计算机图形学 2026 秋季学期 • Final Project 第 13 题

3DGS 质量保持压缩

训练视角可见贡献评分与固定数量恢复优化

从可复现 baseline 到预算、画质与性能的完整对照

组员一	【姓名一待填写】 【学号一待填写】
组员二	【姓名二待填写】 【学号二待填写】
课程 / 班级	【班级或教师要求的信息待填写】
报告日期	2026 年 10 月 21 日
质量结果版本	`d433184 / 0af71e9 / b756c61`
重复计时版本	`2de407a`

摘要结论 在 train 和 truck 两个场景中，贡献度剪枝保留 50% 高斯后恢复 3000 步，相对同样继续训练的完整模型，PSNR 分别低 0.0526 和 0.0085 dB；PLY 文件约减半，三轮独立进程计时的配对加速比分别为 1.456 和 1.408。更激进的 30% 保留率带来约 1.75 倍加速，但画质损失扩大。本项目不主张所有场景都优于不透明度剪枝，也不将微小分差称为显著提升。

核心 CUDA 渲染器直接使用官方实现；项目新增模块和 AI 辅助边界见第7节。

摘要

3D Gaussian Splatting（3DGS）以大量带颜色与透明度的三维高斯表示场景，能够实时合成新视角图像，但高斯数量会带来模型存储和渲染开销。本项目在固定官方 2023 版本上完成 baseline，进一步实现基于训练视角可见贡献的高斯筛选，并在固定数量条件下进行 3000 步恢复优化。评分复用官方可微渲染器的颜色梯度，统计每个高斯经过遮挡后的平均混合权重，不使用测试图像。

实验覆盖 train、truck 两个场景，70%、50%、30% 三档保留率，完整模型、不透明度剪枝、贡献度剪枝及其恢复训练的六组对照。为避免把额外训练收益归因于压缩，每档预算均与同样恢复 3000 步的完整模型比较。性能对 18 个恢复后模型设置进行三轮独立进程复测，轮换执行顺序，报告 PSNR、SSIM、LPIPS、模型大小、渲染耗时和明确口径的显存。

结果表明，50% 保留率在本实验中取得较好的画质与资源平衡；30% 则展示了压缩极限和细节损失。贡献度方法在激进压缩时比不透明度排序更能保持质量，但同数量下未必渲染更快。本项目是课程范围的工程扩展与系统评测，不主张提出了全新的通用压缩理论或复现了原论文全部分数。

关键词：3DGS；高斯剪枝；可见贡献；恢复优化；新视角合成；可复现实验

目录

1 问题定义与项目范围 **3**

1.1 我们要解决什么 3

1.2 课程要求与实现边界 3

1.3 从输入到报告的系统路径 3

2 Baseline：3DGS 在做什么 **4**

2.1 高斯不是一个像素，也不是一个三角形 4

2.2 怎样从椭球得到一张照片 4

2.3 模型如何学出来 4

3 扩展方法：可见贡献剪枝与恢复 **5**

3.1 为什么不能只按不透明度排序 5

3.2 训练视角贡献评分 5

3.3 稳定选择与文件完整性 5

3.4 固定数量恢复优化与公平对照 6

4 实验设置与验证协议 **7**

4.1 数据、环境及实验矩阵 7

4.2 质量与资源指标 7

4.3 核验结果与局限 7

5 定量结果与分析 **8**

5.1 核心结论：中等压缩是更平衡的工作点 8

5.2 完整预算与恢复消融曲线 9

5.3 train：全部质量记录 10

5.4 train 的主要代价 10

5.5 truck：全部质量记录 11

5.6 truck 说明了什么 11

5.7 独立复测性能曲线 12

5.8 最终性能数值与波动范围 13

5.9 预处理与恢复开销 14

6 定性结果与失败案例 **14**

6.1 对比图的选择与阅读方式 14

6.2 train 视角 00014：车体与地面结构 15

6.3 train 视角 00033：底盘细节与残余伪影 16

6.4 truck 视角 00014：木板、轮胎与边缘 17

6.5 truck 视角 00031：主体稳定与纹理损失 18

7 可复现性、来源与贡献说明 **19**

7.1 直接使用与新增实现 19

7.2 关键代码路径 19

7.3 从仓库到结果的运行说明 19

8 局限与结论 **20**

8.1 局限 20

8.2 结论 20

1 问题定义与项目范围

1.1 我们要解决什么

输入是同一场景的多张照片、相机内外参和初始稀疏点云。baseline 先学习一个能从新视角渲染的高斯模型。我们的问题不是如何训练全新的重建网络，而是：**在已有场景模型中删除部分高斯，能否用更少存储与更低渲染成本保持接近原来的质量？**这里的压缩是模型结构压缩，不是把同一个 PLY 文件打成 ZIP。

预算定义为保留比例 r。若原模型有 N 个高斯，则保留

$$K = \max(1, \lfloor rN \rfloor), \qquad r \in \{0.7, 0.5, 0.3\}. \tag{1}$$

例如保留 30% 意味着删除 70%。每个预算都直接从原始完整 30k 模型选择，不是在 50% 模型上二次剪枝，也不是在恢复后的模型上重新评分。

1.2 课程要求与实现边界

课程第 13 题要求解释 Gaussian 参数、投影、alpha 合成及增密/剪枝，并完成一个实质扩展；统一要求还包括可运行系统或可复现实验、技术报告、5–8 分钟演示或现场 Demo 及贡献说明。本项目选择**压缩**作为扩展：新增训练视角贡献评分、稳定 Top-K 选择、固定数量恢复、六组消融与独立重复计时。没有重写 CUDA splatting 内核，也没有实现抗锯齿、动态场景或网格提取。

本报告依据归档的评测记录与渲染图像，使用来源哈希校验保证结果对应关系。完整 PLY、断点和数据集独立于 Git 存储。两个场景的结果不能替代更大规模数据集上的泛化验证。

1.3 从输入到报告的系统路径

阶段	主要输入、操作与输出
Baseline	多视图、相机、稀疏点云；官方优化 30000 步；输出完整 SH3 高斯 PLY。
评分与选择	全训练视角计算可见贡献；与不透明度排序比较；分别保留 70%/50%/30%。
恢复	每个筛选模型和完整对照均继续优化 3000 步；不增密、不删点、不重置透明度。
评测	留出测试视角渲染，计算三种画质指标；记录模型数量、大小、CUDA 事件计时。
复测与核验	18 个最终模型设置各 3 个独立进程；检查来源哈希、逐视角均值与导出图像。

2 Baseline：3DGS 在做什么

2.1 高斯不是一个像素，也不是一个三角形

本节依据原论文与官方实现 [1, 2]。一个高斯可理解为三维空间中的半透明椭球。它有中心 $\boldsymbol{\mu}$、方向与尺度、透明度和随视角变化的颜色。其空间分布为

$$G(\mathbf{x}) = \exp\left[-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\right]. \tag{2}$$

协方差通过旋转与正尺度构造：$\Sigma = RSS^\top R^\top$。实现保存的是对数尺度、未激活透明度和四元数；渲染前使用指数、sigmoid 与归一化，避免直接优化非法参数。颜色使用球谐（SH）系数，以表达从不同方向看同一点时的颜色变化；本实验最终固定为 3 阶。

官方 SH3 PLY 的每行包含 62 个 float32 字段：位置 3、占位法线 3、颜色直流 3、其余 SH 系数 45、透明度 1、尺度 3、四元数 4。法线占位字段不是本项目另学得的表面法线。筛选时整行保留所有字段，不能只导出位置和 RGB，否则不再是同一个可渲染模型。

2.2 怎样从椭球得到一张照片

首先将中心变换到相机坐标并投影到屏幕。对透视投影局部线性化后，可近似得到二维协方差

$$\Sigma_{2D} \approx JW\Sigma W^\top J^\top, \tag{3}$$

其中 W 是视图变换的线性部分，J 是投影在高斯中心附近的 Jacobian。于是每个三维高斯在屏幕上成为一个二维椭圆足迹。

渲染器按屏幕 tile 组织高斯并进行深度排序。对一个像素 p，从近到远混合：

$$\mathbf{C}_p = \sum_i T_{pi}\alpha_{pi}\mathbf{c}_i + T_{p,\text{end}}\mathbf{c}_{bg}, \quad T_{pi} = \prod_{j<i}(1-\alpha_{pj}). \tag{4}$$

α_{pi} 同时包含该高斯透明度和在像素处的二维高斯响应；T_{pi} 表示光线到达它之前还剩多少透过率。被前方遮住的高斯即使本身很不透明，对最终图像也可能几乎没有贡献。官方 CUDA 实现还包含阈值、截断和提前终止等工程处理，本项目复用其实际前向与反向结果。

2.3 模型如何学出来

baseline 从 SfM 稀疏点初始化，随机抽取训练相机，渲染图像并与照片计算损失：

$$\mathcal{L} = 0.8\,\mathcal{L}_1 + 0.2\,(1-\text{SSIM}). \tag{5}$$

反向传播更新位置、尺度、旋转、透明度和 SH 系数。训练中的密度控制会根据梯度和尺度克隆或拆分高斯，并删除低透明度等不合适的高斯，从而把更多容量放在难以重建的区域。本项目 baseline 沿用固定版本的默认密度控制及 SH 升级流程。**baseline 中的动态增密不等于本项目的训练后压缩**：前者为提高拟合能力调整容量，后者在已训练模型上减少容量。

3 扩展方法：可见贡献剪枝与恢复

3.1 为什么不能只按不透明度排序

不透明度剪枝保留 raw opacity logit 最大的 K 行；sigmoid 单调递增，因此在精确数学意义上等价于按激活透明度排序，使用 logit 可避免 sigmoid 浮点饱和造成不必要的并列。该基线实现简单，但只看高斯自身参数，不看是否被遮挡、屏幕覆盖范围或训练相机是否观察到它。这解释了为何它在某些视角分布和激进预算下会删掉重要细节。

3.2 训练视角贡献评分

固定几何、透明度和排序后，式(4)对颜色是线性的。对训练视角 v 中的像素 p 定义 $w_{vpi} = T_{vpi}\alpha_{vpi}$，我们采用跨视角、跨像素平均的混合权重：

$$s_i = \frac{1}{|\mathcal{V}_{train}|} \sum_{v \in \mathcal{V}_{train}} \frac{1}{P_v} \sum_{p=1}^{P_v} w_{vpi}. \tag{6}$$

它近似回答“这个高斯在训练视角的最终图像里占了多少可见权重”，**不等于删除它造成的重建误差**：它不考虑该点颜色与邻近颜色是否相似，也不直接使用 GT 误差。

实现时冻结所有模型参数，将所有高斯的诊断 RGB 颜色设为 $(0.5, 0.5, 0.5)$、背景设为黑色。令 L_v 为渲染 RGB 图像所有元素的均值，则

$$\sum_{c \in \{R,G,B\}} \frac{\partial L_v}{\partial c_{ic}} = \frac{1}{P_v} \sum_{p} w_{vpi}. \tag{7}$$

因此可通过 `torch.autograd.grad(image.mean(), colors)` 提取可见权重，不用另写一套可能与官方不同的可见性计算。诊断颜色只存在于评分过程，不写回高斯 PLY。GPU 梯度为 float32，跨视角累加为 float64，最后按训练视角数平均。

正确性检查包括非负有限值、权重和与 $2\,\mathrm{mean}(I_v)$ 一致，以及首个训练视角上固定索引子集的颜色有限差分检查。评分只使用训练相机，不读取测试 GT，不用测试分数选择高斯。两个场景分别使用全部 263 和 219 个训练视角。

3.3 稳定选择与文件完整性

两种方法均按分数降序稳定排序，分数相同按原始索引打破平局；选中后恢复原始行顺序再保存，减少额外顺序变化带来的深度并列影响。所有 62 个 float32 字段逐项读回比较，保存选择索引及 SHA256。70%、50%、30% 都用原始完整模型和同一份全训练视角评分，不能对每个预算重新使用测试结果调评分。

3.4 固定数量恢复优化与公平对照

删点后，剩余高斯可以调整自身参数来重新分担被删部分的贡献。我们用 fresh Adam 继续优化 3000 步，但禁止增密、剪枝、透明度重置或 SH 升阶。所有组使用相同的训练相机序列（seed=0，逐轮无放回打乱）及相同损失。完整模型也执行同样恢复，避免把“多训练 3000 步”的收益归因于压缩。

参数组	恢复学习率与设置
位置	$1.6 \times 10^{-6} \times$ scene.cameras_extent，常数
SH 直流 / 其余	2.5×10^{-4} / 1.25×10^{-5}
透明度 / 尺度 / 旋转	0.005 / 0.0005 / 0.0001
优化器	Adam，$\epsilon = 10^{-15}$，重新初始化；中断续跑恢复状态
约束	SH3 固定；高斯数固定；只用训练相机；不挑选测试集最优步

从 PLY 加载时需显式设置位置学习率的场景尺度；否则其默认值为零可能使位置参数事实上不更新。本项目对此已有回归测试。断点每 500 步保存，但本报告不把可恢复代码等同于已证明跨所有中断情形逐位一致。

4 实验设置与验证协议

4.1 数据、环境及实验矩阵

使用官方发布的 `tandt_db.zip` 中 train 与 truck 场景，采用官方留出视角规则及固定数据目录。输入的相机排序与每 8 张留出一张测试视角的划分由数据检查器记录。

场景	训练视角	测试视角	实际渲染尺寸	原始高斯数
train	263	38	980×545	1,082,124
truck	219	32	979×546	2,590,158

运行硬件为 NVIDIA GeForce RTX 4090 D（24GB），驱动 550.120，CUDA Toolkit 12.1，PyTorch 2.2.2+cu121，Python 3.10.22，Linux/glibc 2.28。官方代码固定提交：

`2eee0e26d2d5fd00ec462df47752223952f6bf4e`。

黑背景，分辨率参数 -1，baseline 优化 30000 步，恢复后评测 33000 步。

每个场景、每个预算有六组：完整、不透明度、贡献度及三者各加恢复 3000 步。两场景三预算共 36 组质量记录，完整对照在各任务重复保留。质量实验均为单种子，不能将重复对照或多视角当作多个独立训练种子。

4.2 质量与资源指标

使用官方评测脚本对全部留出视角求均值：PSNR 和 SSIM 越高越好，LPIPS 越低越好（官方 VGG 实现）。PSNR 按每张图的 MSE 计算后再跨视角平均，不把所有像素混在一起计算单一 PSNR。同预算完整恢复模型是主要质量对照；原始 30k 完整模型只用于说明额外训练的影响。模型大小统计未压缩二进制 PLY 字节数；MiB 按 2^{20} 字节换算。

渲染时间采用 CUDA 事件，不含模型加载、PNG 编码和磁盘 I/O。最终模型的每个进程先预热 20 帧，再遍历测试集 3 次，因此 train 有 114 个、truck 有 96 个帧时间样本。三个方法执行顺序按轮次循环轮换，18 个模型设置各启动 3 个独立进程，共 54 份样本。报告进程内平均毫秒数在 3 轮之间的中位数及最小/最大值；FPS 为 1000/mean_ms。加速比先按同预算同轮完整模型与压缩模型的均值相除，再对三轮比值取中位数。显存仅指计时范围内 PyTorch 峰值 allocated bytes，不是整卡占用或训练峰值。

4.3 核验结果与局限

重复计时在同一主机、同一次资源分配中完成，以减少硬件和运行条件差异。18 个设置三轮均值的最大相对范围约 0.536%，小于预设 10% 工程检查线。该检查说明本次计时稳定，不是统计显著性检验，也不能代表所有 GPU 上的速度。数据审计核对 54 份计时、36 组逐视角质量均值、18 组恢复训练记录、210 张导出图像哈希和 180 张渲染的像素 PSNR，并核对计时对应的最终 PLY 哈希。SSIM 与 LPIPS 采用 GPU 评测记录；完整审核清单由资产生成脚本写入 `report/generated/audit.json`。

5 定量结果与分析

5.1 核心结论：中等压缩是更平衡的工作点

表中 Δ 均为“贡献度 + 恢复”减“同预算完整 + 恢复”，正 LPIPS 差值代表退化。50% 数据来自此前六组实验，其最终模型与本次复测逐项核对，不将旧单轮 FPS 混入最终表。

场景	保留率	ΔPSNR	ΔLPIPS	加速比	大小比例
train	70%	+0.000828	+0.000450	1.239	0.700
train	50%	-0.052603	+0.006755	1.456	0.500
train	30%	-0.432146	+0.033242	1.751	0.300
truck	70%	+0.000069	+0.000012	1.207	0.700
truck	50%	-0.008457	+0.000355	1.408	0.500
truck	30%	-0.329130	+0.006449	1.746	0.300

70% 保留率的微小正 PSNR 差异远不足以证明质量提升。50% 时两个场景均保持接近完整模型的 PSNR，同时文件减半，稳定获得约 1.4 倍加速。30% 时质量损失扩大，特别是 train 的 LPIPS 增加 0.03324，不能称为无损压缩。50% 是观察完整曲线后的工程推荐，**不是在测试集上选出参数后得到的无偏泛化估计**；报告仍完整展示三档预算。

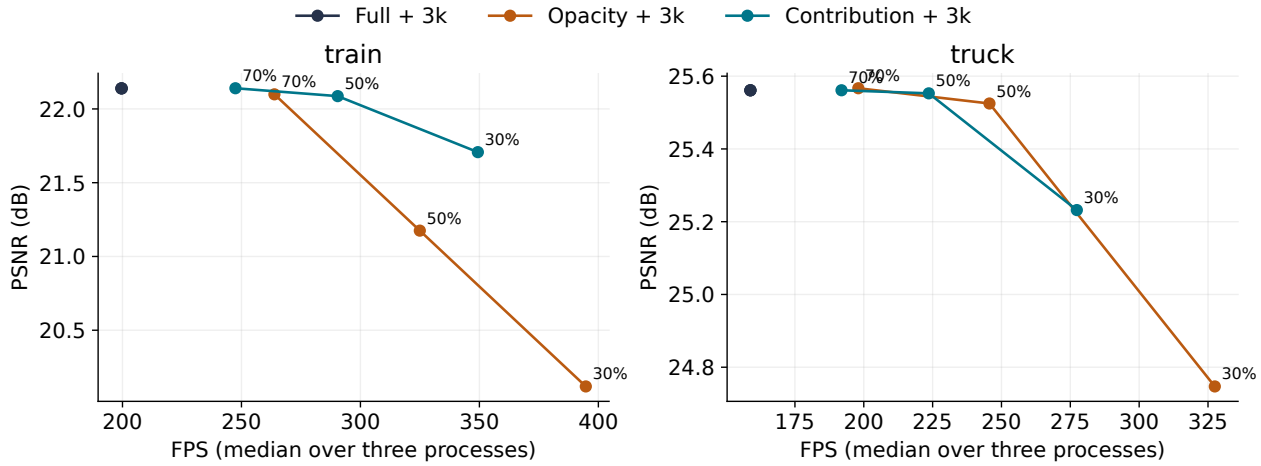


图 1: 质量与速度的折中。质量为全测试视角均值，FPS 为三进程中位数。完整对照是各预算各自的模型，点可能几乎重合。

不透明度方法同数量下通常更快，但在 30% 时质量明显较差；贡献度方法不是每项指标都占优。渲染成本还受高斯屏幕足迹、重叠、可见性和 tile 工作量影响，不能只由数量推断。

5.2 完整预算与恢复消融曲线

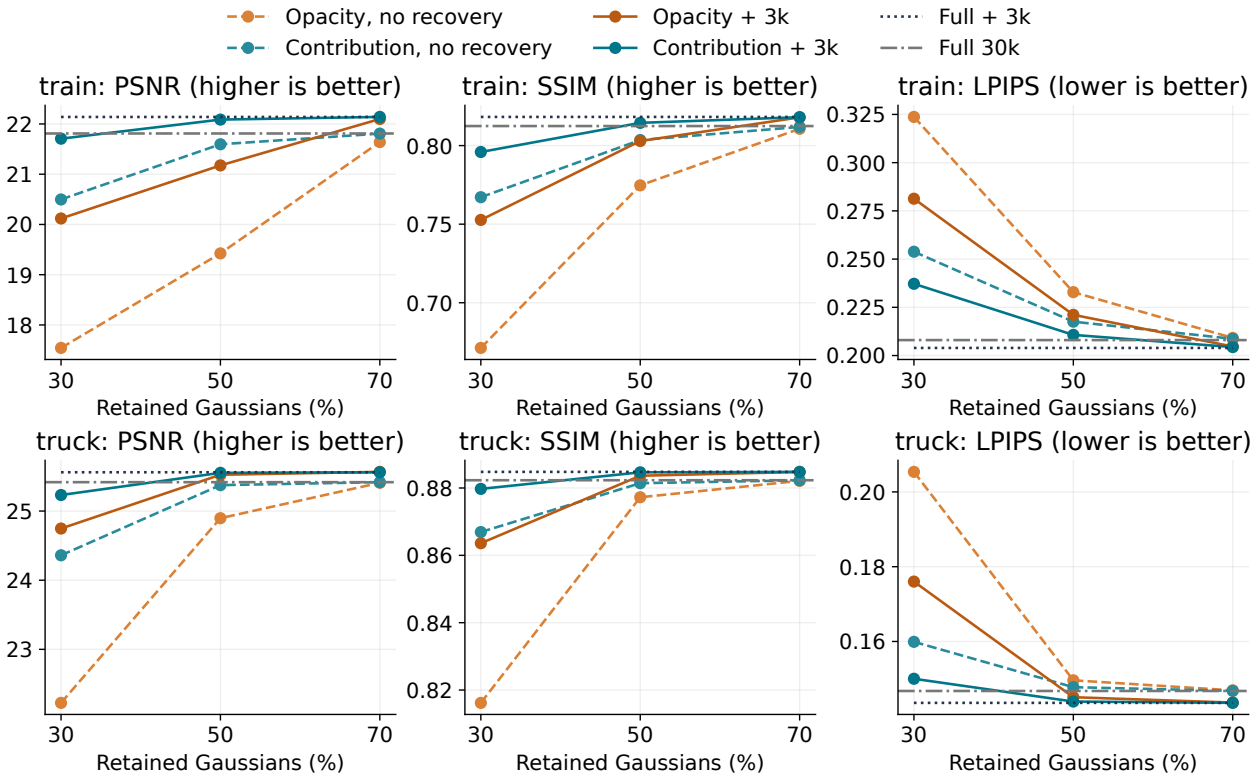


图 2: 两场景完整质量曲线。虚线为不恢复的压缩模型，实线为恢复后模型；完整模型分别保留 30k 和加 3k 两种对照。

train 在 50% 不恢复时，不透明度剪枝 PSNR 为 19.4242，贡献度为 21.5959；加入恢复后分别为 21.1750 与 22.0868，说明评分和恢复都有作用。但完整模型同样从 21.8105 提升到约 22.1394，因此压缩组优于原始 30k 不能被视为压缩提高了画质。

truck 在 50% 恢复后，两种剪枝 PSNR 只相差约 0.0279 dB，贡献度的优势较小；70% 时不透明度恢复组 PSNR 还略高约 0.0056 dB 且渲染更快。这些结果应完整报告，而不是只挑 train 或 30% 预算突出方法优势。在 30% 恢复后，贡献度相对不透明度的 PSNR 优势为 train 1.5881 dB、truck 0.4848 dB，说明在更紧的预算下训练视角信息更有价值，但仍不能外推到所有场景。

5.3 train：全部质量记录

“预算”表示任务的压缩目标；完整模型行始终保留 100%，并非完整模型也被剪到该预算。每个值来自 38 个测试视角；重复完整对照的微小差异来自独立恢复运行。

预算	方法	PSNR	SSIM	LPIPS	高斯数	PLY MiB
70%	完整	21.810524	0.812434	0.208012	1,082,124	255.94
70%	不透明度	21.639704	0.810675	0.209124	757,486	179.16
70%	贡献度	21.807867	0.811881	0.208663	757,486	179.16
70%	完整 + 恢复	22.139441	0.818239	0.203945	1,082,124	255.94
70%	不透明度 + 恢复	22.098965	0.817956	0.204686	757,486	179.16
70%	贡献度 + 恢复	22.140268	0.818075	0.204394	757,486	179.16
50%	完整	21.810524	0.812434	0.208012	1,082,124	255.94
50%	不透明度	19.424191	0.774688	0.232863	541,062	127.97
50%	贡献度	21.595947	0.803524	0.217566	541,062	127.97
50%	完整 + 恢复	22.139402	0.818250	0.203952	1,082,124	255.94
50%	不透明度 + 恢复	21.175035	0.802914	0.221105	541,062	127.97
50%	贡献度 + 恢复	22.086800	0.814473	0.210707	541,062	127.97
30%	完整	21.810524	0.812434	0.208012	1,082,124	255.94
30%	不透明度	17.542755	0.671249	0.323712	324,637	76.78
30%	贡献度	20.497404	0.767161	0.253825	324,637	76.78
30%	完整 + 恢复	22.139246	0.818241	0.203946	1,082,124	255.94
30%	不透明度 + 恢复	20.118963	0.752698	0.281305	324,637	76.78
30%	贡献度 + 恢复	21.707100	0.795929	0.237188	324,637	76.78

5.4 train 的主要代价

30% 贡献恢复组 PSNR 为 21.7071，比原始 30k 完整模型仅低 0.1034 dB，但比公平的同预算完整恢复对照低 0.4321 dB。因此若只展示前一个差距，会掩盖压缩的真实机会成本。视觉上应特别检查轮组、链条和高频地面纹理，而不是只观察远景轮廓。

5.5 truck：全部质量记录

每个值来自 32 个测试视角。truck 比 train 包含更多高斯，但不意味着其质量或帧率应按同一比例变化。

预算	方法	PSNR	SSIM	LPIPS	高斯数	PLY MiB
70%	完整	25.417843	0.882305	0.146760	2,590,158	612.60
70%	不透明度	25.408709	0.882120	0.146921	1,813,110	428.82
70%	贡献度	25.418036	0.882290	0.146788	1,813,110	428.82
70%	完整 + 恢复	25.561222	0.884786	0.143570	2,590,158	612.60
70%	不透明度 + 恢复	25.566845	0.884764	0.143655	1,813,110	428.82
70%	贡献度 + 恢复	25.561291	0.884786	0.143582	1,813,110	428.82
50%	完整	25.417843	0.882305	0.146760	2,590,158	612.60
50%	不透明度	24.896971	0.877248	0.149610	1,295,079	306.30
50%	贡献度	25.374870	0.881424	0.147770	1,295,079	306.30
50%	完整 + 恢复	25.561270	0.884787	0.143576	2,590,158	612.60
50%	不透明度 + 恢复	25.524910	0.883636	0.145088	1,295,079	306.30
50%	贡献度 + 恢复	25.552813	0.884649	0.143931	1,295,079	306.30
30%	完整	25.417843	0.882305	0.146760	2,590,158	612.60
30%	不透明度	22.225498	0.816180	0.205360	777,047	183.78
30%	贡献度	24.360453	0.866919	0.159902	777,047	183.78
30%	完整 + 恢复	25.561100	0.884788	0.143576	2,590,158	612.60
30%	不透明度 + 恢复	24.747164	0.863592	0.176016	777,047	183.78
30%	贡献度 + 恢复	25.231970	0.879736	0.150025	777,047	183.78

5.6 truck 说明了什么

该场景在 70% 和 50% 预算下相对宽松，恢复训练能弥补大部分不透明度剪枝损失；贡献评分在 30% 时优势更明显。结论应是“收益随场景与预算变化”，而不是“复杂评分总比简单评分好”。所有组对远处文字和纹理的重建仍可能存在误差，这些误差中有一部分来自 baseline 本身。

5.7 独立复测性能曲线

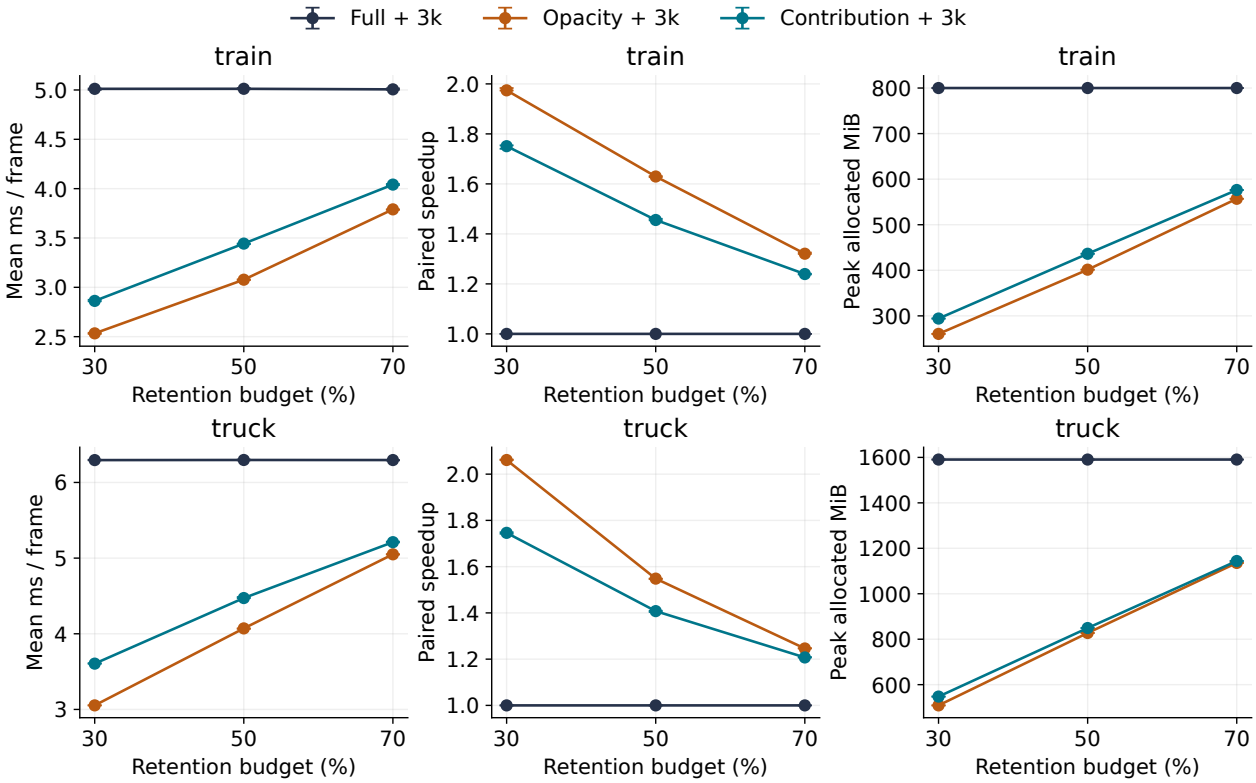


图 3: 最终恢复模型的重复计时与显存。误差棒为三进程均值的最小/最大范围，不是置信区间；波动很小时误差棒可能难以分辨。完整行不随目标预算剪枝。

在 50% 预算下，贡献度方法在 train 和 truck 的配对加速比分别为 1.4556 与 1.4077；30% 时分别为 1.7508 与 1.7462。同数量不透明度方案的速度更高，进一步说明可见贡献更大的高斯也可能带来更多实际光栅化工作。显存随压缩总体降低，但本表只统计 PyTorch 分配峰值；不能据此声称 GPU 整卡显存同比下降。

5.8 最终性能数值与波动范围

以下所有方法均已恢复 3000 步，单位 MiB 采用 2^{20}。每档预算完整对照独立保留。

train

预算	方法	中位 ms	范围 ms	FPS	加速比	显存 MiB
70%	完整	5.0063	5.0019–5.0163	199.75	1.000	799.98
70%	不透明度	3.7900	3.7849–3.7907	263.85	1.321	556.93
70%	贡献度	4.0411	4.0352–4.0473	247.46	1.239	576.23
50%	完整	5.0120	5.0073–5.0196	199.52	1.000	799.98
50%	不透明度	3.0773	3.0728–3.0808	324.96	1.630	401.38
50%	贡献度	3.4430	3.4371–3.4432	290.44	1.456	436.29
30%	完整	5.0116	5.0015–5.0143	199.54	1.000	799.98
30%	不透明度	2.5334	2.5281–2.5393	394.73	1.974	260.48
30%	贡献度	2.8624	2.8586–2.8739	349.36	1.751	294.15

truck

预算	方法	中位 ms	范围 ms	FPS	加速比	显存 MiB
70%	完整	6.2962	6.2909–6.2971	158.83	1.000	1590.56
70%	不透明度	5.0501	5.0461–5.0525	198.02	1.246	1135.69
70%	贡献度	5.2107	5.2068–5.2186	191.91	1.207	1144.05
50%	完整	6.2970	6.2925–6.3027	158.80	1.000	1590.56
50%	不透明度	4.0711	4.0642–4.0722	245.64	1.548	827.34
50%	贡献度	4.4722	4.4702–4.4862	223.60	1.408	849.11
30%	完整	6.2954	6.2928–6.3002	158.85	1.000	1590.56
30%	不透明度	3.0537	3.0535–3.0544	327.47	2.061	508.74
30%	贡献度	3.6053	3.6028–3.6122	277.37	1.746	547.53

性能测量未包含评分、剪枝、模型加载、恢复优化或写 PNG 开销，因此这里的加速是最终模型的渲染收益，不是整个处理管线的端到端加速。三个进程不构成三个训练种子，对细小质量差异不作显著性结论。

5.9 预处理与恢复开销

评分与恢复均有一次性计算成本。下表直接使用日志中的计时字段，评分排除逐视角 checkpoint I/O，恢复排除 checkpoint I/O；不包括环境安装、模型加载、最终保存、评测和所有外部调度等待。因此它们不是完整墙钟耗时，也不能简单与 FPS 相抵消来估算端到端收益。

场景	预算	共享评分 s	完整恢复 s	不透明度恢复 s	贡献度恢复 s
train	70%	3.115	58.458	46.264	48.385
train	50%	3.115	58.463	36.075	41.357
train	30%	3.115	58.503	27.955	34.027
truck	70%	3.245	89.141	69.387	70.463
truck	50%	3.245	89.098	54.223	58.337
truck	30%	3.245	89.103	38.284	44.994

全训练视角评分每场景只计算一次，三预算复用同一数组；表中共享评分数值重复列出是为方便阅读，不能累加成三次评分。不透明度排序不需要这一评分过程，但也有未在此独立计时的 PLY 读取、排序和保存开销。恢复学习率较小，耗时只是短程优化成本；原始 30k baseline 训练成本不属于压缩额外开销。

6 定性结果与失败案例

6.1 对比图的选择与阅读方式

使用预先固定的 train 视角 00014/00033、truck 视角 00014/00031，同时展示 GT、完整恢复、不透明度恢复与贡献度恢复。每张图第一行展示 50% 预算的完整视图并标出黄色裁剪框，第二行显示同一裁剪区域，第三行在同一位置显示 30% 预算。第三行的完整恢复对照来自 30% 任务自身，不偷换为 50% 任务模型。裁剪坐标在资产脚本中固定并一致应用于各方法，未做锐化、降噪或颜色修正。

这四张图是诊断视角，不是随机抽样画质盲评。图像缩放可能隐藏像素级差异，因此既展示局部也保留全测试视角指标；PNG 和矢量图文件随报告源码保存。以下不把 baseline 中已有的地面条纹、远景模糊或几何不完整都归因于压缩。

6.2 train 视角 00014：车体与地面结构

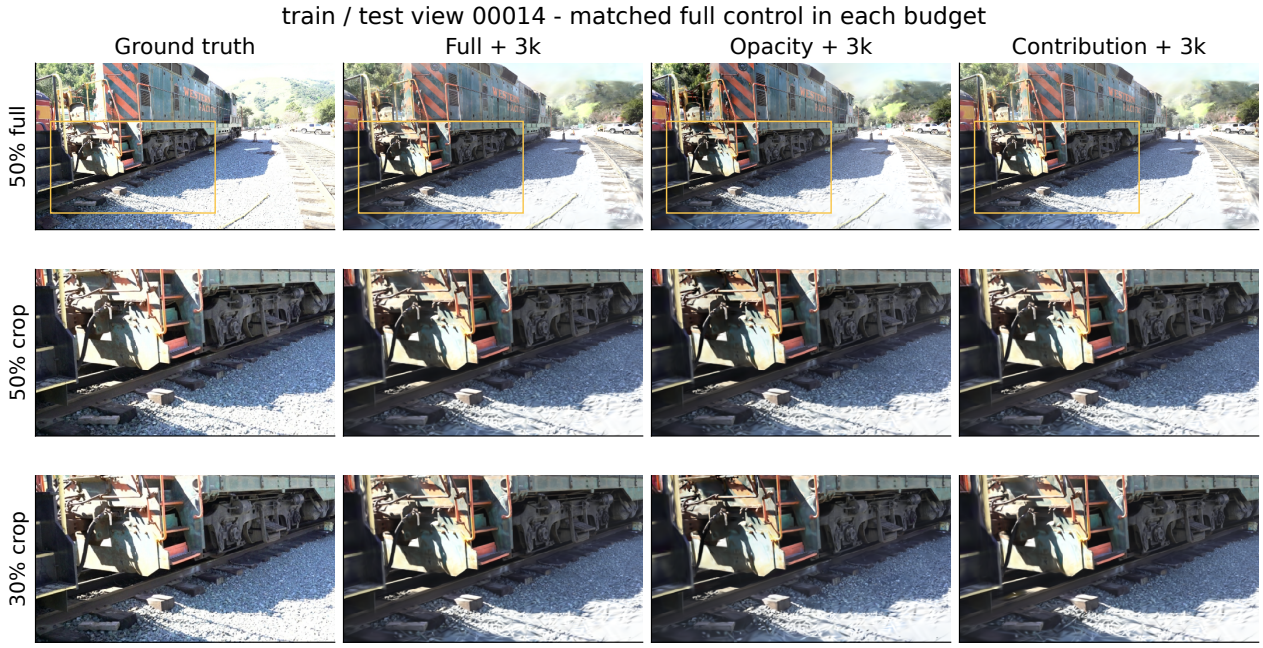


图 4: train 固定视角 00014。上行 50% 全图，中行 50% 局部，下行 30% 同位置局部。

重点看车体边缘、轮轨附近和地面高频结构。贡献度排序优先保留训练视角中实际可见的部分，但不保证高频细节不损失。从 50% 降到 30% 时，剩余高斯需要覆盖更多原本由细小高斯表示的区域，恢复优化可以减少平均误差，却不必然恢复所有细节。不透明度的较高 FPS 必须与其画质退化同时解释。

6.3 train 视角 00033：底盘细节与残余伪影

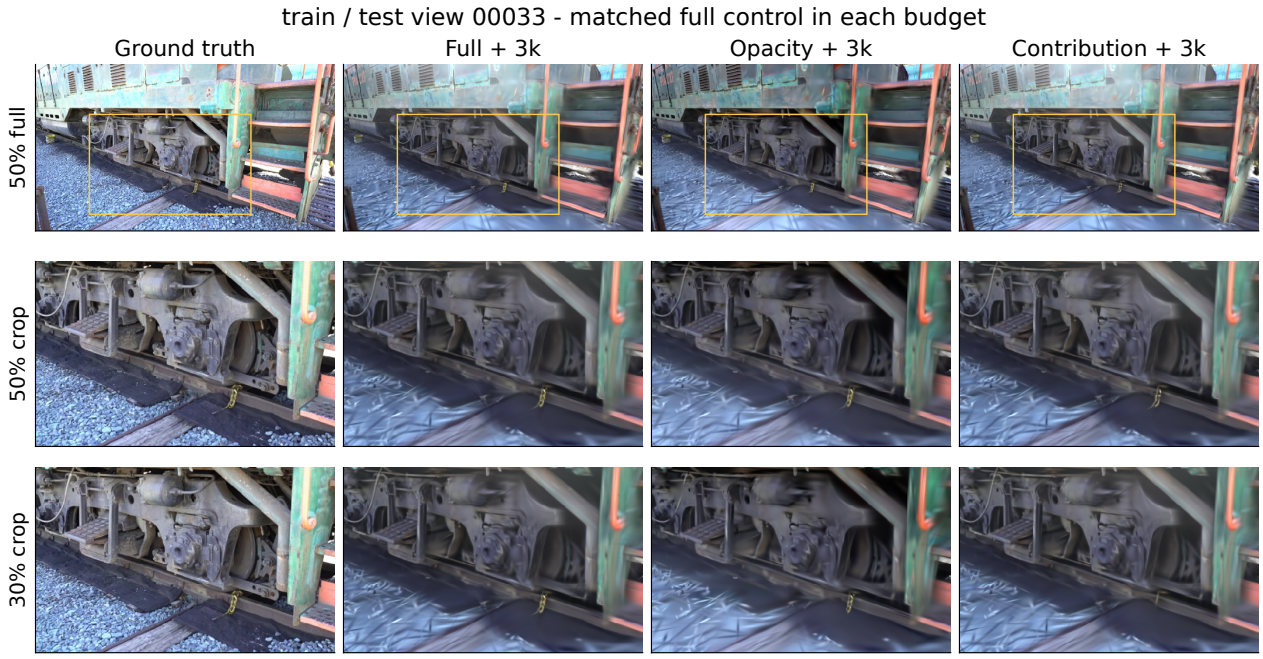


图 5: train 固定视角 00033，包含底盘、轮组及地面失败区域。各预算使用自己的完整恢复对照。

该视角显示车轮、管线、链条等细节在压缩后的平滑与丢失。地面尖锐条纹在完整恢复模型中已经存在，因此主要是 baseline 的重建失败区域，不是贡献评分方法新增的全部误差。本项目恢复阶段不改变拓扑、不增加高斯，无法凭空补回缺少的几何表达容量。

6.4 truck 视角 00014：木板、轮胎与边缘

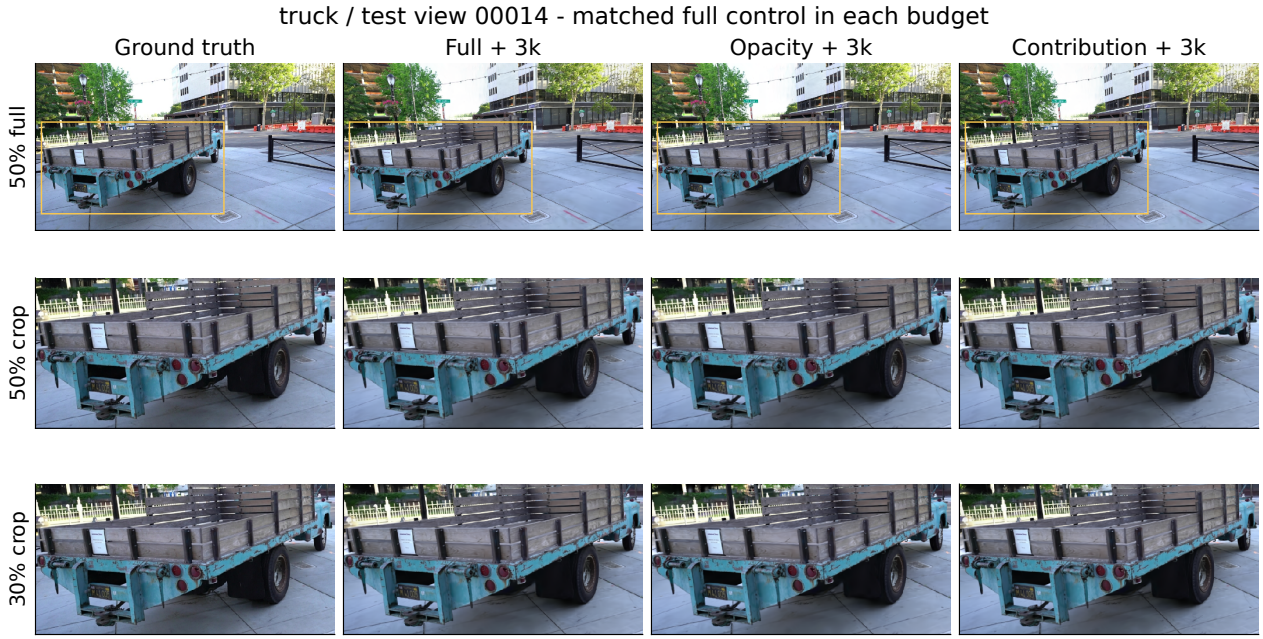


图 6: truck 固定视角 00014。主体结构保留较好，局部细节仍需结合 GT 判断。

50% 预算下两种恢复方案总体都接近完整模型，这与两者 PSNR 仅有小差距一致。30% 预算提供更强存储压缩，但车体边缘、木纹和轮胎区域的细节容量更紧张。不能因为一张全图看起来相近就认定两方法等价，也不能仅凭小分差宣称贡献方法显著优越。

6.5 truck 视角 00031：主体稳定与纹理损失

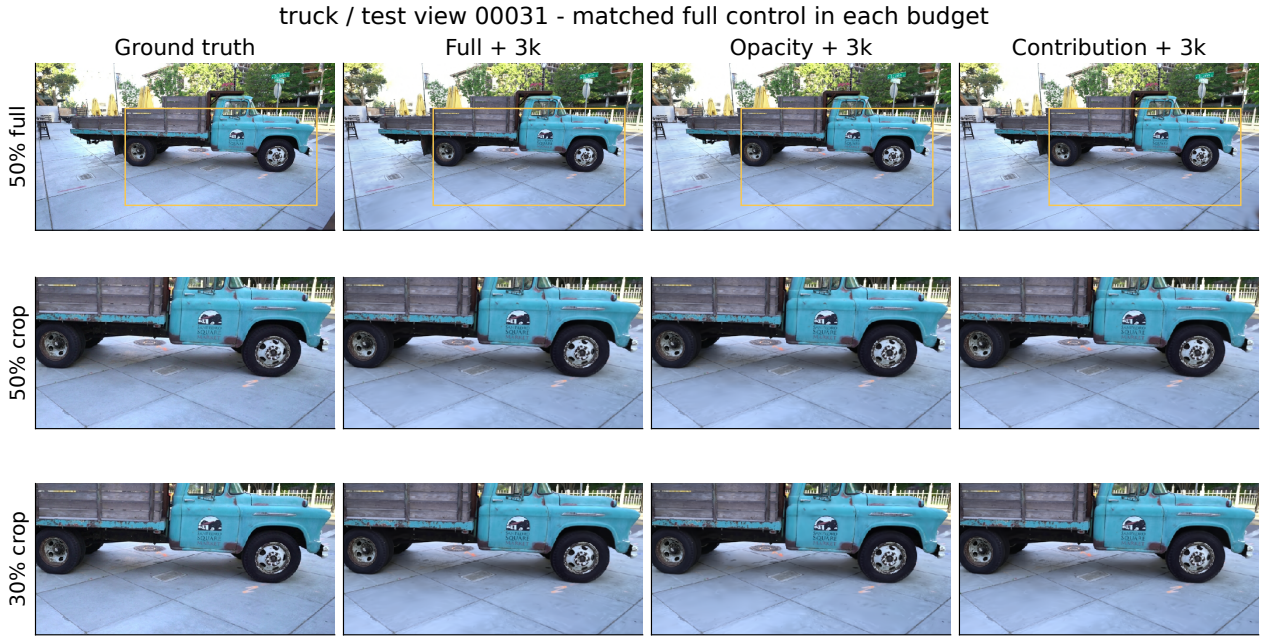


图 7：truck 固定视角 00031。全图用于观察结构，局部用于观察车体和轮组细节。

车身、木板、轮廓整体仍可辨认，地面与小纹理存在平滑。即使贡献 30% 方案明显优于不透明度 30% 的平均指标，它仍比同预算完整恢复模型低 0.3291 dB。项目的价值是建立可解释的画质–资源折中，而不是把压缩结果包装为比完整模型更准确的重建。

7 可复现性、来源与贡献说明

7.1 直接使用与新增实现

直接使用的官方模块	高斯表示、相机加载、baseline 训练、密度控制、CUDA rasterizer、kNN 初始化、渲染及质量评测；固定官方版本，保留许可证。
本项目新增模块	训练视角颜色梯度评分、稳定选择与 PLY 字段校验、固定数量恢复与完整对照、跨场景/预算编排、来源哈希关卡、计时复测、结果导出与审计图表。
AI 辅助范围	使用 Codex 辅助工程代码、测试、资料讲解与报告整理。

成员一	【姓名一、学号一、实际负责内容与贡献说明待填写】
成员二	【姓名二、学号二、实际负责内容与贡献说明待填写】

7.2 关键代码路径

- 官方 baseline：`vendor/gaussian-splatting/train.py`、`scene/gaussian_model.py` 及 `gaussian_renderer`。
- 剪枝：`scripts/prune_ply.py`；评分：`scripts/contribution_phase3.py`。
- 六组与预算：`scripts/recovery_phase4.py`、`scripts/budget_phase5.py`；恢复训练器各自冻结版本。
- 独立计时：`scripts/repeat_timing.py`、`scripts/benchmark_repeat_worker.py`。
- 报告资产：`scripts/build_report_assets.py`，只读取导出 JSON/PNG，不运行 GPU。

7.3 从仓库到结果的运行说明

详细安装与数据条款见 `docs/CLOUD.md` 及 README；CUDA 扩展须在匹配 PyTorch 和 Toolkit 的环境编译。原始数据与大模型不在 Git 中，仅拉取 Git 结果可以重建报告图表，但不能直接重新渲染场景。完整实验研究记录见`docs/COMPRESSION_EXPERIMENT.md`，运行命令集中在 README。

报告资产由 `scripts/build_report_assets.py` 生成，随后对 `main.tex` 执行两次 XeLaTeX 编译。数据、模型与评测使用同一实验配置，来源清单记录各阶段输入哈希。资产生成需要 NumPy、Pillow 和 Matplotlib；LaTeX 使用 XeLaTeX、ctex 与 Fandol 字体。报告依赖使用独立环境；实验配置与来源哈希用于确保各阶段结果的一致性。

8 局限与结论

8.1 局限

1. 只验证了两个静态场景和一张 GPU，质量实验为单种子；不宣称普适最优、统计显著或复现原论文全部成绩。
2. 可见权重不是删除误差，也没有显式颜色残差、几何语义或屏幕工作量约束；复杂评分不保证更快。
3. 恢复只优化剩余参数，不增密，无法修复全部几何缺失；训练相机覆盖之外的细节仍可能被低估。
4. 本报告只提供渲染器 CUDA 事件 FPS 及 PyTorch 分配峰值，不是端到端交互系统帧率或整卡显存。
5. 预算推荐来自观察测试结果，保留全曲线并披露选择过程；若要进一步调参，应另设验证划分。

8.2 结论

本项目完成了官方 baseline 的可复现实验，并加入可见贡献剪枝与固定数量恢复这一实质扩展。公平完整对照、跨场景多预算消融以及稳定重复计时共同支持：50% 是当前两个场景上较平衡的工程工作点，30% 能进一步降低资源开销但需要接受可测量的质量损失。

参考文献

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM Transactions on Graphics, 42(4), 2023. DOI: 10.1145/3592433.

[2] Graphdeco/Inria. Official Gaussian Splatting implementation. `https://github.com/graphdeco-inria/gaussian-splatting`. 本项目固定提交见第 4 节与 `baseline.lock.json`，许可证保持原样。

[3] 3DGS official project resources. `https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/`. 本项目使用其发布的 train/truck 输入数据；数据与图像版权归原提供方。

[4] 计算机图形学 2026 秋季学期，Final Project 选题与统一要求，第 2、11、16 页。

[5] 本项目导出证据：`results/`下六个质量实验目录与 `timing-repeat-job4380448`；自动核验来源清单见 `report/generated/audit.json`。